Introducing APD: a novel multivariate public key cryptosystem

Borja Gómez
kub0x@elhacker.net

February 14, 2020

Abstract

This paper introduces a new trapdoor function that is used to construct the cryptosystem called Affine Polynomial Decomposition (APD), which performs encryption and signing. Its invertibility is commented as depends on the Subset Sum problem and the Degree Decomposition problem, which is introduced by the author. Besides, the cryptosystem's security is analyzed from the perspective of the computational hardness of solving a system of quadratic equations over a field with q elements plus the Isomorphism of Polynomials with two secrets. In addition, a formal cryptanalysis of the multivariate scheme Hidden Irreducible Polynomials is given, due to APD being an extended variation of this. In the end, a novel technique for Key Recovery Attacks is presented, from the perspective of the private polynomial $\mathcal{F}$ being publicly known, which reduces the security assumption of **IP2** to **IP1** when the initial affine transform S has a special structure.

1 Introduction

In Oct. 2019 the author published a novel *MPKC* cryptosystem based on a bijective trapdoor involving the multiplication of irreducible polynomials over F_2, called *Hidden Irreducible Polynomials* (**HIP**). Short time after, C. Eder shown that a linear algebra attack is always possible. The author's conclusion here is that his scheme is neither based on **IP2** nor **IP1**. As a consequence, a linear transformation T is applied on the quadratic polynomial map $\mathcal{F}$ resulting in $P(X)$. As $P(X), F(X, Y)$ are known, an attacker recovers T successfully. Linear Algebra attacks are introduced and commented along the paper, specially in the section of the cryptanalysis of **HIP** (see REF) where a general case cryptanalysis is given. All the algorithms and techniques provided by the author in this research are implemented on Mathematica and are accessible in REF.

2 Affine Polynomial Decomposition: A new scheme based on ideas of HIP

The author in [1] made a remark about a variant of the scheme, where instead of using one linear transformation matrix, it uses two affine transformations $S, T \in GL(n, F_2) \times F_q^n$, one at the beginning and the other at the end of the public map construction. This variation changes the behavior of the whole cryptosystem first because the factorisation now doesn't give the original irreducible polynomials, but a product of distinct ones that can be or not irreducible and secondly, the plaintext tuple $\overrightarrow{x}$ is not required to hold two irreducible polynomials, but any polynomial, with a minor restriction as further seen.Thereof, the idea wasn't addressed in the **HIP** paper as it would result in the construction of a new cryptosystem, which wasn't the aim in the research. The goal of this section is to build a cryptosystem based on this idea.

2.1 Description of the cryptosystem

BigField family of schemes relies on operating with elements from an extension field $\mathbb{E}$ of $\mathbb{F}_q$ of degree n. For example in the schemes C^* and **HFE**, at some point polynomials are reduced modulo an irreducible polynomial over F_q of degree n. In the other hand, an extension field $\mathbb{E}$ is not required in the design of **APD** cryptosystem, same as schemes belonging to the Intermediate Field family REF like: UOV, TTM, STS. SimpleMatrix, $\cdots$). The elements lie in $F_q[y]$: the univariate polynomial ring over F_q, so the polynomial $\mathcal{F}(X, Y)$ multiplies two polynomials but doesn't reduce the result modulo an irreducible polynomial.

$$\varphi_i : F_q^i \mapsto F_{q^i}, \quad (x_1, \cdots, x_k) \mapsto \sum_{i=1}^{i} x_i y^{i-1} \tag{1}$$

$$S : F_q^{n+1} \mapsto F_q^{n+1}, \quad \overrightarrow{x} \mapsto A_1 \cdot \overrightarrow{x} + \overrightarrow{v_s} \tag{2}$$

$$\phi : F_q^{n+1} \mapsto F_{q^k} \times F_{q^k}, \quad (x_1, \cdots, x_{n+1}) \mapsto (\varphi_k(x_1, \cdots, x_k), \varphi_k(x_{k+1}, \cdots, x_{n+1})) \tag{3}$$

Notice the dimension in (5) as we have $2k = n+1$ variables it outputs a $n+1$-tuple of affine linear polynomials in $(x_1, \cdots, x_{n+1})$. This dimension is necessary as it's partitioned into two polynomials of k elements in (6), so degree $k-1 = \frac{n-1}{2}$.

$$\mathcal{F} : F_{q^k} \times F_{q^k} \mapsto F_{q^n}, \quad (p_1(y), p_2(y)) \mapsto (p_1 p_2)(y) \tag{4}$$

$$T : F_q^n \mapsto F_q^n, \quad \overrightarrow{x} \mapsto A_2 \cdot \overrightarrow{x} + \overrightarrow{v_t} \tag{5}$$

$$P(X) = T \circ \varphi_n^{-1} \circ \mathcal{F} \circ \phi \circ S(\overrightarrow{x}) \tag{6}$$

Then in (7), $\mathcal{F}$ gives $p_3(y)$ with degree $n-1$ and $n+1$ variables. As p_3 is an element of F_{q^n}, represent it as a n-tuple over F_q then apply the affine transformation in (8). The whole process is represented in (9).

2.2 Analysing the private polynomial $\mathcal{F}$

Initially on **HIP**, when inverting $P^{-1}(Y) = X$ the owner of the construction faces the problem of computing $\mathcal{F}^{-1}(p_3(y)) = (p_1(y), p_2(y)))$ which is equal to polynomial factorisation over F_q. Since Bob selects two irreducible polynomials, concatenates both coefficient tuples and evaluates $P(X) = Y$. Then p_3 is the product of two irreducible polynomials in variables $(x_1, \cdots, x_{n+1})$ and we recover the plaintext tuple. However, **using an initial affine** transformation changes the behavior of the trapdoor, as we don't certainly know the degree of these two polynomials that give $p_3(y) = F \circ \phi \circ S(\overrightarrow{x})$, moreover, they can be *composite* in the sense that contribute multiple factors with distinct degrees because $p_1(y)$ and $p_2(y)$ now have affine coefficients in variables $(x_1, \cdots, x_{n+1})$. In this section we cover two cases, depending on the previous knowledge of the degree of polynomial factors $p_1(y), p_2(y)$. Both cases contain instances of the subset sum problem: the first having just one instance as the degrees are previously known and the second one having multiple instances because the degrees of $p_1(y)$ and $p_2(y)$ are unknown. The author calls the combination of polynomial factorisation and Subset Sum problem as the *Degree Decomposition Problem.*

2.2.1 Subset Sum Problem

Before entering the description of the *Degree Decomposition Problem* it is necessary to explain how the Subset Sum problem is involved in this problem. Let n be a positive integer and $\mathcal{H} = (h_1, \cdots, h_r) \in \mathbb{Z}^{+^r}$ be a r-tuple of arbitrary values. Our goal is to enumerate all the possible partitions of $\mathcal{H}$ that sum up to n. The naive strategy clearly has a computational cost of 2^n as the algorithm ranges from 0 to $2^n - 1$. This is equal to trying all the different combinations of the form:

$$s_i = \sum_{j=0}^{r} \alpha_j h_j, \quad \alpha = (\alpha_1, \cdots, \alpha_r) \rightarrow \alpha = \text{ToBinaryList}(i, r)$$

Where $\text{ToBinaryList}(i, r)$ gives the binary representation of i in a r-tuple and s_i is a sum on $\mathcal{H}$ that's equal to the scalar or dot product of $\mathcal{H} \cdot \alpha$, as it counts h_i in the sum whenever $\alpha_i = 1$. An instance of the Subset Sum Problem appears in the decryption process of the proposed **APD** cryptosystem, which will be solved in the next subsection to determine two polynomials that satisfy a degree relation.

2.2.2 Degree Decomposition Problem

As seen in section 3.2, $p_3(y)$ could have degree *less than* $n-1$ so we must find out the correct degree of polynomials $p_2(y), p_3(y)$. Remember that $\mathcal{F}$ multiplies two polynomials with maximum degree $k-1$, so both $p_1(y), p_2(y)$ have degree less or equal than $k-1$. It is mandatory to separate the process in two cases:

This problem is strongly related to the Subset Sum problem on the set of degrees found in the canonical factorisation of $p_3(y)$ composed of irreducible polynomials over F_q.

Let's construct the set of degrees needed for finding two polynomials $p_1(y), p_2(y)$ with degree less or equal to $k-1$ where their degrees sum up to $Deg(p_3(y))$. The direct way is to write the

canonical factorisation of $p_3(y)$ on r irreducible polynomials over F_q of arbitrary degree that sum up to $Deg(p_3(y))$:

$$p_3(y) = (p_1 p_2)(y) = \prod_{i=1}^{r} f_i(y)^{e_i}$$

Where $e = (e_1, \cdots, e_r) \in \mathbb{Z}^{+^r}$ is the exponent tuple. Hence, there are r irreducible polynomials over F_q in the decomposition where the polynomial f_i appears e_i times in the canonical factorisation. This is strongly related to the cycle type representative of an integer partition of $Deg(p_3(y))$ by the relationship:

$$\sum_{i=1}^{r} Deg(f_i(y)) * e_i = Deg(p_3)$$

$$type(\lambda) = (Deg(f_1) * e_1, \cdots, Deg(f_r) * e_r)$$

Then $type(\lambda)$ groups multiple equal partition elements, here the degrees of f_i's, into an unique element, up to multiplicity. This is, when $\lambda_i = \lambda_j$ write $2 \times \lambda_i$. However, the cycle-type partition $type(\lambda)$ is not enough to construct both $p_1(y), p_2(y)$ in some cases, as selecting an irreducible factor's degree in $type(\lambda)$ gives degree $Deg(f_i) * e_i$ but not $Deg(f_i)$ solely, so there would be some instances of the subset sum problem without solution, this is, either impossible to sum to $k-1$ when $Deg(p_3(y)) = n-1$ or impossible to sum to $k-j$ when $Deg(p_3)(y) < n-1$. The workaround here is to declare the extended partition

$$\lambda \vdash Deg(p_3) = (\overbrace{Deg(f_1), \cdots, Deg(f_1)}^{e_1 \text{times}}, \cdots, \overbrace{Deg(f_r), \cdots, Deg(f_r)}^{e_r \text{times}})$$

Obviously, the partition $type(\lambda)$ would give solutions in some instances of the subset sum plus the partition length of $type(\lambda)$ is r and the partition length of $\lambda \vdash Deg(p_3(y))$ is $\sum_{i=1}^{r} e_i$, which is greater.

Taking this into account, to obtain the right polynomials $p_1(y), p_2(y)$ we must find **two independent sequences** in the degree partition λ such that $seq_1 + seq_2 = n-1, \quad seq_1, seq_2 \leq k-1$ (these always exist). This way, we have two polynomials such that $Deg(p_3(y)) = Deg(p_1'(y)) + Deg(p_2'(y)$. Then check if $p_3(y) = (p_1' p_2')(y)$. When the condition is satisfied, we've found the right polynomials. However, applying $S^{-1}(\phi^{-1}(p_1', p_2'))$ for recovering $\overrightarrow{x}$ can give a wrong $n+1$-tuple, as the univariate polynomial quotient ring $F_q[y]$ is commutative, there's a chance that p_1', p_2' must be switched in position.

In the case that $Deg(p_3(y)) = Deg(p_1'(y)) + Deg(p_2'(y)$ but $p_3(y) \neq (p_1' p_2')(y)$ either $Deg(p_1'(y)) < Deg(p_2'(y))$ or $Deg(p_2'(y)) < Deg(p_1'(y))$, meaning that the map ϕ doesn't partition $\overrightarrow{x}$ into two blocks of equal length k but into **distinct** lengths l_1, l_2 such that $l_1 + l_2 = 2k = k+1$. Conclude that among all the valid degree sequences that sum up to $k-1$, there only exist one that satisfies $\overrightarrow{x} = S^{-1}(\phi^{-1}(p_1', p_2'))$.

2.2.3 Fixing degree of affine polynomials

1. If $Deg(p_3(y)) = n-1$ it is clear that $Deg(p_1(y)) = Deg(p_2(y)) = k-1$, so we construct two polynomials of equal degree with the help of $p_3(y)$'s factorisation as seen further. A valid solution for decryption exists in the solution set of the Subset Sum problem instance.

2. In the case that $p_3(y)$ has degree lower than $n-1$ denote the degree difference by $j = n - 1 - Deg(p_3(y))$. We must construct two polynomials with degree less than $k-1$ where their degrees sum up to $n-j$. There could be different pairs $(p_1(y), p_2(y))$ that satisfy the sum, i.e $Deg(p_1(y)) = k-j, Deg(p_2(y)) = k-1, \quad Deg(p_1(y)) =$. The answer is that there are j polynomials that satisfy the degree equality, up to commutativity. The proof can be given, for an arbitrary j as:

$$\forall i: \quad Deg(p_1(y)) = k-j \wedge Deg(p_2(y)) = k-i, \quad 1 \leq i \leq j$$

3. The most noticeable difference between 1. and 2. is that in the first the degree of $p_3(y)$ is known to be $n-1$. As none of the degrees of $p_1(y)$ and $p_2(y)$ can't exceed $k-1$ we're restricted to have only two polynomials of degree $k-1$. In the second method we must enumerate construct a list of j polynomials where $j = n-1-Deg(p_3(y))$, as there are j ways to construct polynomials that sum up to $n-j$ with degree less than $k-1$. **Conclude** that the first method faces a single instance of the Subset Sum Problem where the second faces j instances. At this moment, the author only considers following the analysis of the first method. This is, the degree of $p_3(y) = \varphi \circ F \circ \phi \circ S(\overrightarrow{x})$ is always $n-1$. This is possible, by modifying some parameters on the initial phase of the cryptosystem.

As the requirement now implies that $Deg(p_1(y)) = Deg(p_2(y)) = k-1$, for that, set the two rows in A_1 that contribute x_k, x_{n+1} and the two positions k and $n+1$ in the scalar vector $\overrightarrow{v_s}$ in the following way where $\alpha_i, \beta_i, \gamma_i \in F_q$:

$$
\begin{aligned}
(A_1)_k &= [\overbrace{\alpha_i, \cdots, \alpha_{k-1}}^{[1,k-1]}, 1, \overbrace{\alpha_{k+1}, \cdots, \alpha_{n+1}}^{[k+1,n+1]}] \\
(A_1)_{n+1} &= [\overbrace{\beta_1, \cdots, \beta_n}^{[1,n]}, 1] \\
\overrightarrow{v_s} &= (\gamma_1, \cdots, \gamma_{k-1}, 0, \cdots, \gamma_n, 0) \\
\overrightarrow{x} &= (x_1, \cdots, x_{k-1}, 1, \cdots, x_n, 1)
\end{aligned}
$$

Note how in the input $n+1$-tuple $\overrightarrow{x}$ variables x_k, x_{n+1} has been set to 1. After applying the affine transformation, partition it into two polynomials such that $\phi(S(\overrightarrow{x})) = (p_1(y), p_2(y))$ and clearly $Deg(p_1(y)) = Deg(p_2(y)) = k-1$. This is a consequence of the affine transform fixing x_k in position k and x_{n+1} in position $n+1$. As **these are always** 1 we know beforehand that these two affine polynomials have degree $k-1$. Note that the quadratic public map $P(X)$ now has $n-1$ variables instead of $n+1$ since x_k, x_{n+1} have value 1.

When decrypting a ciphertext factorise $p_3(y)$ into the canonical decomposition into irreducible polynomials over F_q, as seen in 3.2.2, obtain the degree partition and construct the solution set of the Subset Sum problem. For every solution construct $p_1(y), p_2(y)$ and test decryption.

2.3 Is the APD polynomial in the family of HFE polynomials?

The first noticeable difference is that an **HFE** polynomial has only one input variable X whereas the **APD** polynomial has two variables X, Y. The second one is that **HFE** polynomials take advantage from the action of the Frobenius Automorphism being linear, this is let $X = (\alpha_0 + \alpha_1 y + \cdots + \alpha_n y^{n-1}) \in F_{q^n}$. Then $X' \equiv_{f(x)} X^{q^i} \equiv_{f(x)} \sum_{i=0}^{n-1} \alpha_i y^{iq^i} \equiv_{f(x)} \sum_{i=0}^{n-1} \alpha'_i y^i$. Then X' is an affine polynomial in variables $(\alpha_0, \cdots, \alpha_n)$. Then the simplest version of an **HFE** polynomial $HFE(X) = X^{q^a+q^b}$ computes the polynomial multiplication of two affine polynomials derived from X since $X^{q^a+q^b} = X^{q^a} X^{q^b}$. Thereof, we could write $X^{q^a} X^{q^b} = XY$ and try to see if the **APD** polynomial $\mathcal{F}(X,Y) = XY$ can be represented as an **HFE** polynomial. By inspection it results that the answer is positive, despite of having one variable versus two, the **APD** polynomial can be represented as an **HFE** polynomial composed of a product of two distinct *Linearised Polynomials* over F_{q^n}.

2.3.1 Linearised polynomials over F_{q^n}

The author discovered that the set of *Linearised Polynomials* with coefficients lying on F_{q^n} is isomorphic to the matrix ring $\mathcal{M}_{n\times n}(F_q)$, where n is the degree of the extension field over F_q and both structures having cardinality $q^{n^2} = q^{2n}$. Later the author realized that this result is well known in literature see REF. This connection allows us to represent $X^{q^a} X^{q^b}$ as the product of two polynomials in F_{q^n} using linear transformation matrices $A_1, A_2 \in F_q^{n\times n}$ as follows: $\varphi(A_1 \cdot \overrightarrow{x})\varphi(A_2 \cdot \overrightarrow{x})$, where the action of φ sends a n-tuple to a polynomial of degree $n-1$.

2.3.2 Computing the equivalent HFE polynomial

Since we want that $HFE(X||Y) = APD(X,Y) = X^{q^a} X^{q^b} = XY$ the transformation matrices A_1, A_2 must separate X into X, Y where $X = \sum_{i=1}^{n} x_i y^{i-1} \quad Y = \sum_{i=k}^{n} x_i y^{n-1}$. A natural way to define both transformation matrices is to use a block matrix definition:

$$
A_1 = \begin{pmatrix} I & 0 \\ 0 & 0 \end{pmatrix}
$$
$$
A_2 = \begin{pmatrix} 0 & I \\ 0 & 0 \end{pmatrix}
$$

Then, $A_1 \overrightarrow{x} = (x_1, \cdots, x_k, 0, \cdots, 0)$ and $A_2 \overrightarrow{x} = (y_1, \cdots, y_k, 0, \cdots, 0)$ where $\overrightarrow{x} = (x_1, \cdots, x_k, y_1, \cdots, y_k)$. This proves that there exists an equivalent **HFE** polynomial for the **APD** polynomial $\mathcal{F}(X,Y)$ since two linear transformation matrices A_1, A_2 exists that satisfy the product XY. However, the author is unaware of a method for computing the polynomial form, this is, find two linearised polynomials such that $HFE(X) = L_1(X||Y)L_2(X||Y) = XY$. In mathematical terms we could think on knowing the morphism $\mathcal{L}_n \to \mathcal{M}_{n\times n}(F_q)$ because it is easy to obtain the linear transformation matrix associated to a Linearised Polynomial. But knowing $\mathcal{M}_{n\times n}(F_q) \to \mathcal{L}_n$ seems hard, this is obtaining the Linearised

Polynomial associated to a linear transformation matrix. If such a method exists, $\mathcal{F}^{-1}(X,Y)$ could be computed by polynomial factorisation or root finding over F_{q^n} i.e by Berlekamp's only if the **HFE** polynomial defining $\mathcal{F}$ is monic and has maximum degree between 100 and 127.

2.4 Monomial Basis and Public Key Size

Recall that **HIP** has monomial basis size of k^2 for $k = 1 + \frac{n-1}{2}$ as polynomial multiplication mixes univariate monomials of two k-tuples with the condition that monomials of the same tuple are not combined. Moreover, T is a **linear** transformation matrix, not an affine transform, this is, the monomial basis lacked from the scalar value 1. These questions have been addressed in this chapter. Including an initial affine transformation S changes totally the monomial basis found in the construction, and this result is important for the public key bit size. The effect of S can be understood from the viewpoint of the input on $\mathcal{F}$. Two polynomials $p_1(y), p_2(y)$ are multiplied, since the two k-tuples defining these polynomials have affine variables in $(x_1, \cdots, x_{n+1})$, the previous restriction of not combining monomials of the same tuple is gone. Then $p_3(y)$ could have all the possible $1 + \frac{n(n+1)}{2}$ monomials. Concretely, from the point of Oil-Vinegar, the private polynomial $\mathcal{F}(X,Y)$ is an Oil-Vinegar polynomial where the monomials found are of type $x_i y_j$, matching our argument. Thereof, the effect of the affine transformation S in the public key bit size can be given in three cases. Let $n' = n - 1$ since x_k, x_{n+1} are equal to 1 plus now the sets Oil and Vinegar variables have $k' = k - 1$ elements.

$$\begin{cases} S = \begin{pmatrix} A_1 & A_2 \\ 0 & A_3 \end{pmatrix} + \overrightarrow{v_s}, S = \begin{pmatrix} 0 & A_1 \\ A_2 & A_3 \end{pmatrix} + \overrightarrow{v_s}, \\ S = \begin{pmatrix} A_1 & 0 \\ A_2 & A_3 \end{pmatrix} + \overrightarrow{v_s}, S = \begin{pmatrix} A_1 & A_2 \\ A_3 & 0 \end{pmatrix} + \overrightarrow{v_s} \\ |M_B| = 1 + \frac{1}{2}n'(n'+1) - \frac{1}{2}k'(k'+1) + k' \end{cases} \tag{1}$$

$$\begin{cases} S = \begin{pmatrix} A_1 & A_2 \\ A_3 & A_4 \end{pmatrix} + \overrightarrow{v_s} : |M_B| = 1 + \frac{1}{2}n'(n'+1) \end{cases} \tag{2}$$

$$\begin{cases} S = \begin{pmatrix} A_1 & 0 \\ 0 & A_2 \end{pmatrix} + \overrightarrow{v_s} : |M_B| = 1 + k'^2 + n' \end{cases} \tag{3}$$

All the matrices $A_i \in F_q^{n \times n}$ and vectors $\overrightarrow{v_s} \in F_q^n$ **have the form described in section** 3.2.2, plus univariate monomials x_k, x_{n+1} and the quadratic monomial $x_k x_{n+1}$ are eliminated from the monomial basis.
In (1) S introduces monomials of type $(x_i, y_i, y_i y_j, x_i y_j)$ or $(x_i, y_i, x_i x_j, x_i y_j)$. So the polynomials on the public keys $P(X)$ never contain Oil-Oil or Vinegar-Vinegar terms for the case $i \neq j$. In (2) S introduces all type of monomials $x_i y_j, x_i x_j, y_i y_j$ where $i = j$ is possible, so the public key has complete size. In (3) S introduces monomials x_i, y_j and $x_i y_j$, so the public key has only Oil-Vinegar variables plus linear Oil and linear Vinegar. This case is equal to applying two different affine transformations one to the set of Oil variables and the other to Vinegar variables, then merging both vectors into one of dimension n. Then the public key size can be ordered as (3) < (1) < (2) where in (2) and (3) an attacker could substitute for variables y_i or x_i, depending if there are no Oil-Oil or Vinegar-Vinegar monomials in the public key. This way he would obtain an overdetermined linear equation system on variables x_i or y_i where the coefficient matrix has dimension $n \times k'$. In the next section of this article, we will see that when working with digital signatures and the Oil-Vinegar approach, the only affine transform S that can be selected is (2) as it wouldn't result in a determined linear equation system.

2.5 Digital Signatures

The **APD** cryptosystem aims to encipher a plaintext $\overrightarrow{x}$, where the ciphertext is $P(\overrightarrow{x}) = \overrightarrow{y}$. When signing, the output $P(\overrightarrow{x}) = \overrightarrow{y}$ is the digest of a message m combined with a salt s. The owner must recover the n-tuple $\overrightarrow{x} = (x_1, \cdots, x_{k-1}, 1, \cdots, x_n, 1)$ such that $P(\overrightarrow{x}) = H(m||salt)$. The usual procedure to achieve this is given as:

$$\begin{aligned} &Alice : \overrightarrow{y} \leftarrow H(m||s), \quad \overrightarrow{x} = P^{-1}(\overrightarrow{y}) \\ &Alice \xrightarrow{(\overrightarrow{x}, m, s)} Bob \\ &Bob : P(\overrightarrow{x}) = y = H(m||s) \end{aligned}$$

An attacker can generate arbitrary pairs $P(\overrightarrow{x}) = \overrightarrow{y}$ but yet he cannot know the message m that is needed to satisfy the equality $H(m||s) = \overrightarrow{y}$. This attack is arbitrary and non-interesting for an attacker, because an attacker wants to forge signatures on concrete values i.e his own crafted message. For that he must invert the public map construction, facing the $\mathcal{MQ}$ problem along with the $\mathcal{IP}$ with two secrets.

2.5.1 APD and Signatures

In the **APD** cryptosystem, the public quadratic map has n polynomials in $n-1$ variables as initially we had $n+1$ variables, however, $x_k = x_{n+1} = 1$ is set so the output polynomial $p_3(y) = \mathcal{F}(p_1(y), p_2(y))$ has degree $n-1$. As a consequence, whenever Bob evaluates the public key, the corresponding polynomial $p_3(y) = \varphi^{-1} \circ T^{-1}(Y)$ has always degree $n-1$. This is, every image Y on the quadratic public map $P(X)$ leads to a polynomial $p_3(y)$ with degree $n-1$. When dealing with digital signatures, the major concern is that $\overrightarrow{y} = H(m||s)$ could not be an image, so $P(\overrightarrow{x}) \neq \overrightarrow{y}$ meaning that $p_3(y)$ has not degree $n-1$. This behavior can be addressed by trying different salts until hit a $p_3(y)$ with degree equal to $n-1$. Thus, the algorithm for performing digital signatures is slightly different from the classic one found in section 3.4:

1. Alice computes $\overrightarrow{y} = H(m||s_i)$. Then she obtains $p_3(y) = \varphi^{-1} \circ T^{-1}(\overrightarrow{y})$ and checks if $Deg(p_3) = n-1$. If this is correct, she knows that $\overrightarrow{y}$ is in the image of P. If the answer is negative, she selects a new salt s_i.
2. Once Alice knows that $Deg(p_3(y)) = n-1$, computes $\overrightarrow{x} = S \circ \phi^{-1} \circ \mathcal{F}^{-1}(p_3(y))$.
3. Alice sends $(\overrightarrow{x}, m, s)$ to Bob.
4. Bob checks if $P(\overrightarrow{x}) = H(m||s)$.

3 Security of the APD cryptosystem

The security of multivariate schemes is usually based on the computational hardness assumptions of the $\mathcal{IP}$ problem and $\mathcal{MQ}$ problem. There are well known attacks found in literature TODOREFS. This section focus on the security impact of the private polynomial $\mathcal{F}$ being publicly known as it's a necessary condition.

3.1 Impact of $\mathcal{F}$ being publicly known

Both schemes **HIP** and **ABD** use the same univariate private polynomial $p_3(y) = \mathcal{F}(p_1(y), p_2(y))$ as the private polynomial of the public map construction. This is a requirement as $\mathcal{F}$ represent the polynomial multiplication over a finite field F_q. The inverse of the function is the *polynomial factorisation* which has complexity TODOREF. A notable consequence is that $\mathcal{F}$ is publicly known and an attacker could figure out how to use this for his advantage.

3.1.1 Perspective from $\mathcal{IP}$ (Isomorphism of Polynomials)

The affine transform S is responsible of making the input-output of $\mathcal{F}$ unknown so the cryptanalysis of TODOREF is not applicable. At this point, we have two ways of defining the effect of S in the cryptosystem, however, both are equivalent.
The first one has been already commented. First $S(\overrightarrow{x})$ is computed, then partitioned into two polynomials over F_q that are multiplied resulting in $p_3(y)$. It results that the computation of $p_3(y)$ is equivalent to $p_3'(y)$ by *Polynomial Affine Equivalence* (**PAE**) REF or *IP with one secret* (**IP1**). TODOREF. Let $p_3'(y)$ represent the polynomial system that defines the output of $\mathcal{F}$. Recall that $\overrightarrow{x} = (x_1, \cdots, x_k)$ and $\overrightarrow{y} = (x_{k+1}, \cdots, x_{n+1})$. Compute $p_3'(y) = F(\varphi(\overrightarrow{x}), \varphi(\overrightarrow{y}))$. It results that $p_3'(S(\overrightarrow{x'})) = p_3(y)$ where $\overrightarrow{x'} = \overrightarrow{x}||\overrightarrow{y}$ (the concatenation of both plaintext tuples). This is because the quadratic polynomial $p_3'(y)$ that outputs from $\mathcal{F}$ is transformed to $p_3(y)$ by an affine change of variables $x_1, \cdots, x_{n+1}$. The most significant effect from $\mathcal{F}$ being publicly know is that an attacker can construct $p_3'(y)$. If there exists a way to recover T, the attacker recovers $\overrightarrow{p_3} = T^{-1}(P(X)) = \mathcal{F}(\phi(\mathcal{S}(\overrightarrow{x}))$, in polynomial form $p_3(y)$. Finally, he solves the instance of **IP1** such that $p_3'(S(\overrightarrow{x})) = p_3(y)$ recovering S. In the strange case that he finds out S but not T, he could transform obtain $p_3(y) = \mathcal{F}(\phi(S(\overrightarrow{x})))$. Now, by the linear algebra attacks introduced in section REF he can recover T.

Conclude that that the scheme must include two invertible affine transforms S, T. In case T is known the attacker can reduce **IP2** to **IP1** as $\mathcal{F}$ is publicly known. In the other hand, knowing S but not T leads the attacker to recovering T by mounting a linear algebra attack as he knows the input-output on the last transformation T. Then a scheme where $\mathcal{F}$ is publicly known, must rely in the **IP2** assumption, setting up S, T secretly. This also explains why the cryptosystem **HIP** is vulnerable.

3.1.2 Linear Algebra Attacks

Rewriting the public key as another quadratic system Moreover, we could instantiate multiple multivariate schemes that share the same monomial basis with **ABD**, in order to have the same dimension on the public key matrix representation as **ABD** does. This way, we could answer in polynomial time if M_i, M_P are related by a linear transformation, where M_P is the public matrix representation of an instance of **ABD** and M_i is the public key matrix representation of the multivariate scheme instance i. Both matrices must have same dimension and be in the same monomial basis, as already said.

In addition, instead of asking whether two quadratic systems of equations are related by a linear transform, we could compute an arbitrary quadratic system of equations by selecting a random linear transformation from $GL(n, F_q)$. Let $C \in GL(n, F_q)$ then $C \cdot M = M_P$ results in the following system $A \cdot x = \overrightarrow{b_{M_P}}$ such that $A \in F_q^{(n \times |M_B|) \times (n \times |M_B|)}$ and $x \in F_q^{(n \times |M_B|) \times 1}$. These linear methods are entirely bounded by the public key size and the number of equations. Normally $n = 128, |M_B| = \frac{n(n+1)}{2}$ over F_2 so the linear equation system has always approximately 2^{20} equations and 2^{14} or 2^{20} variables depending on the method.

4 Key Recovery Attacks

4.1 Introduction

The technique introduced here is *only applicable* when the private polynomial map $\mathcal{F}$ is known. It demonstrates that the scheme is vulnerable depending on the selected initial affine transformation S, where the vulnerability leads the attacker to recover T. Recall that the short version of an usual public map construction based in **IP2** is $P(X) = T \circ F \circ S(X)$. It can be written $P(X) = T \circ \mathcal{A}(X)$ where $\mathcal{A}(X) = F \circ S(X)$ is the quadratic map that results from making an affine substitution in variables $x_1, \cdots, x_n$ in the quadratic polynomial map of $\mathcal{F}$ where $x_i = \alpha_{n+1} + \sum_{i=1}^{n} \alpha_i x_i$ and $\alpha_i \in F_q$. The *key concept* is to separate or partition variables into k sets to maintain in $\mathcal{A}(X)$ some of the monomials found in $F(X)$. This is, to make some monomials invariant by the action of the initial transformation S. Some working examples are given at the end of the section using **HFE** polynomials and the **APD** polynomial.

4.2 Description of the attack

The attacker knows $F(X), P(X)$ and the initial set of variables $\overrightarrow{x} = (x_1, \cdots, x_n)$. The public key can be represented in terms of Linear Algebra by:

$$T \cdot M_{\mathcal{A}(X)} = M_{P(X)}$$

The knowledge of $\mathcal{F}$ does not help at a first glance, as $M_{\mathcal{A}(X)}$ should be a different matrix than $M_{\mathcal{F}}(X)$. But what if $M_{\mathcal{A}(X)}$ fixes columns found in $M_{\mathcal{F}(X)}$? In this occasion, reordering the columns or the monomial basis, we'd have that $M_{\mathcal{A}(X)}$ embeds a submatrix of $M_{\mathcal{F}(X)}$, this is, some columns remain invariant in $M_{\mathcal{A}(X)}$ by the affine change of variables in $\mathcal{F}(X)$. Call this invariant submatrix $M'_{\mathcal{F}(X)}$. Seen as a matrix, $M_{\mathcal{A}(X)} = \begin{pmatrix} M'_{\mathcal{F}(X)} & M_k \end{pmatrix}$ where M_k is an arbitrary $F_q^{n \times (|M_B| - \frac{1}{2}\lambda_i(\lambda_i+1))}$. Clearly, we state that the public key matrix $M_{P(X)}$ contains the linear transformation $T \cdot M'_{\mathcal{F}(X)}$ over F_q as a submatrix. As we know $M'_{\mathcal{F}(X)}$ we know T by linear algebra attacks in REF. This attack can be effective only when the affine transformation S transforms $\mathcal{F}(X)$ to $\mathcal{A}(X)$ making some monomials invariant in every quadratic polynomial in $\mathcal{A}(X)$.

4.3 Generalisation of the attack

Let $n \in Z^+, q = p^k, \quad k \geq 1$ and $\overrightarrow{x} = (x_1, \cdots, x_n)$. Define a partition $\lambda \vdash n$ with partition length k, then $\lambda = (\lambda_1, \cdots, \lambda_k)$. Then, partition $\overrightarrow{x}$ into k-sets where

$$\overrightarrow{x_i} = (x_{\sum_{j=1}^{i-1} \lambda_j}, \cdots, x_{\sum_{j=1}^{i} \lambda_j})$$

From these k sets of variables, we want to make monomials $x_i x_j$ found in every polynomial $p_i(X) \in \mathcal{F}(X)$ invariant in $\mathcal{A}(X)$ where $x_i, x_j \in \overrightarrow{x_r}$, this is the monomial $x_i x_j$ is a combination of monomials of the r-th set exclusively. For that, monomials not composed of elements of $\overrightarrow{x_r}$ cannot be substituted by a combination of monomials of $\overrightarrow{x_r}$ as this would introduce new monomials $x_i x_j$ where both are in $\overrightarrow{x_r}$ and not making these invariant. An example for the reader is given by partitioning $\overrightarrow{x}$ into two sets of variables (i.e Oil and Vinegar) $\overrightarrow{x_1} = (x_1, \cdots, x_k), \overrightarrow{x_2} = (y_1, \cdots, y_k)$. Then if we want to maintain monomials $x_i x_j$ we fix $y_i = \sum_{j=1}^{k} A_{i+k+1,j} y$ and $x_i = x_i + \sum_{i=1}^{k} A_{i,j+k+1} y_k$. Then clearly $x_i x_j$ remains invariant and $y_i y_j$ introduces new $y_i y_j, x_i y_j$ monomials but maintaining $x_i x_j$ invariant, monomials $x_i y_j$ also introduces new $y_i y_j, x_i, y_j$ monomials but maintaining $x_i x_j$ monomials. Then $\mathcal{A}(X)$ would maintain monomials $x_i x_j$ found in every $p_i(X) \in \mathcal{F}(X)$ invariant and this is equal to both matrix representations having at least $\frac{1}{2}k(k+1)$ equal columns. The general description doesn't rewrite variables of distinct sets with distinct symbols, which can be a bit tedious when handling monomials. Let's give the complete algorithm that recovers T entirely when S is selected to have a special form:

1. The variable tuple $\overrightarrow{x} = (x_1, \cdots, x_n)$ and a partition $\lambda \vdash n$ with partition length k such that $\exists \lambda_r \in \lambda : \frac{1}{2}\lambda_r(\lambda_r + 1) \geq n$. This requirement is necessary as $\frac{1}{2}\lambda_r(\lambda_r + 1)$ is the quantity of monomials $x_i x_j$ where $x_i, x_j \in \overrightarrow{x_r}$. Then the invariant quantity must be *greater or equal* to n for the Linear Algebra attack to work. Then $\overrightarrow{x}$ is partitioned into k-sets where

$$\overrightarrow{x_i} = (x_{\sum_{j=1}^{i-1} \lambda_j}, \cdots, x_{\sum_{j=1}^{i} \lambda_j})$$

2. The affine transform S is selected such that for every $p_i(X) \in \mathcal{F}(X)$ it maintains every monomial $x_i x_j$ invariant in $p_i(X) \in \mathcal{A}(X)$ where $x_i, x_j \in \overrightarrow{x_r}$. Then $S \in F_q^{n \times n}$ is a block matrix

composed of $k(k-1)$ matrices such that

$$S = \begin{array}{|c|c|c|c|} \hline I_{\lambda_1} & A_{1,2} & \cdots & A_{1,k} \\ \hline 0 & A_{2,2} & \cdots & A_{2,k} \\ \hline \cdots & \cdots & \cdots & \cdots \\ \hline 0 & A_{k,2} & \cdots & A_{k,k} \\ \hline \end{array}$$

where $A_{i,j} \in F_q^{\lambda_i \times \lambda_j}$.

(a) The horizontal lines represent the partition λ_i such that it defines λ_i variables of the set $\overrightarrow{x_i}$. There are $k-1$ matrices A per row, since there are k partitions, the total number of matrices is $k(k-1)$

(b) Matrices $A_{i,j}$ are rectangular when $\lambda_i \neq \lambda_j$, this is the case when $\lambda \vdash n$ is *heterogeneous*.

(c) When $\lambda \vdash n$ is *complete homogeneous* $A_{i,j}$ is a square matrix and in this case we can calculate the number of matrices S that make monomials of one set $\overrightarrow{x_r}$ invariant.

3. Set the monomial basis $\overrightarrow{m_b}$ such that the first $m = \frac{1}{2}\lambda_r(\lambda_r+1)$ elements are monomial combinations from $\overrightarrow{x_r}$. As a consequence of this order or fixation, matrices $M_{\mathcal{F}(X)}$ and $M_{\mathcal{A}(X)}$ have the first m columns equal, this is, the action of S on the set of all variables $\overrightarrow{x} \in F_q^n$ makes some monomials invariant by the conditions defined above. The public key matrix representation $M_{P(X)}$ is rewritten in the new basis, then as we know that $T \cdot M_{A(X)} = M_{P(X)}$ holds, construct new relations:

$$M'_{F(X)} = M'_{A(X)}$$
$$T \cdot M'_{F(X)} = M'_{P(X)}$$

Where $M'_{F(X)}$ is the submatrix $(n \times m)$ found in $M_{F(X)}$. The same submatrix appear in $M_{A(X)}$ thus if we let $M'_{P(X)}$ to be the $(n \times m)$ submatrix in $M_{P(X)}$ we know the *input-output* of the transformation and being able to recover T.

(a) The point here is that the attacker doesn't know the pair $\mathcal{A}(X), M_{\mathcal{A}(X)}$ but he's able to recover T because the submatrix $M'_{\mathcal{F}(X)}$ is embedded on $M_{\mathcal{A}(X)}$.

4.4 Enumerating dangerous matrices

There are two types of cases for counting matrices S with this form. The first being when $\lambda \vdash n$ is *complete homogeneous* and the other when it's *heterogeneous*. Let's comment the first one:

Let $\lambda \vdash n$ with partition length k such that $\lambda_i = \lambda_j \quad \forall i,j : 1 \leq i \leq j \leq k$. Clearly, we've that $k \mid n$ and $\lambda_i = \frac{n}{k}$, plus matrices $A_{i,j}$ found in S are square matrices. Recall that there exists a necessary condition of λ_i having monomial combinations *greater or equal* than n (see REF). In this case, the condition is that $\frac{\frac{n}{k}(\frac{n}{k}+1)}{2} \geq n$. Then we must calculate the greatest partition number r that satisfies the necessary condition. Every matrix S has k choices of fixing invariant monomials, one per set of variables. Then the formulae to calculate how many S can be given in terms of the factorisation of n

4.4.1 Prime Power case

Let $n = l^{k_1}$ and $q = p^{k_2}$ where $k_1 > 1 \quad k_2 \leq 1$. Given that r is the greatest integer such that $\frac{1}{2}\frac{n}{l^r}(\frac{n}{l^r}+1) \geq n$, we have r ways to partition n into sets of variables, such that a block matrix S can be constructed, which fixes monomial combinations of one set of variables in $\mathcal{A}(X)$ when acting on $\mathcal{F}(X)$. Concretely, the number of such matrices is:

$$\sum_{i=1}^{r} l^i |GL(\frac{n}{l^i}, q)|^{l^i(l^i-1)}$$

For example, take $l = p, q = 2, n = 2^7 = 128$. Then the greatest partition number to satisfy the condition $\frac{\frac{n}{l^r}(\frac{n}{l^r}+1)}{2} \geq 128$ is $r = 3$. Then the bit size of the space of matrices S is:

$$\log_2(\sum_{i=1}^{3} 2^i |GL(\frac{2^7}{2^i}, 2)|^{2^i(2^i-1)}) = 1111111 \text{ bits}$$

As S is supposed to have $\log_2(|GL(128,2))| = 111$ of *"security"* it's reduced to $a-b$ bits. In multivariate schemes, the number of variables n fixes the dimension of the vector space F_q^n. Whenever n is a prime power, this analysis can be applied. As the reader can see there exists a noticeable amount of matrices S that satisfy the requirements defined in this section, but the probability of selecting such a S for this case $q=2, n=128$ is still negligible concretely of 111.

4.4.2 Distinct prime factors

Let $n = l_1^{e_1} \cdots l_s^{e_s}$ and $q = p^k, \quad k \geq 1$. Let $h \mid n$, then h is a factor of n. Then clearly $\lambda \vdash n$ has partition length h where $\lambda_i = \frac{n}{h}$, so every set of variables has length $\frac{n}{h}$. If the requirement $\frac{1}{2}\frac{n}{h}(\frac{n}{h}+1) \leq n$ is satisfied, add h to the set of factors $\mathcal{H}$. Once all possible factors of n have been tested, the formulae for determining the number of matrices S is given by:

$$\sum_{i=1}^{|\mathcal{H}|} h_i |GL(\frac{n}{h_i}, q)|^{h_i(h_i-1)}, \quad h_i \in \mathcal{H}$$

4.4.3 Heterogeneous case

Here n can be *prime or composite*. Recall that in the heterogeneous case the property $\forall \lambda_i, \lambda_j \in \lambda : \lambda_i \neq \lambda_j$ is satisfied, so there exists at least one *partition entry* λ_i that is different from other. As seen in REF, this case is special as block matrices found in S are rectangular matrices of dimension $\lambda_i \times \lambda_j$. The requirement is that the partition must contain a *partition entry* λ_r such that $\frac{1}{2}\lambda_r(\lambda_r+1) \geq n$, so the Linear Algebra attack works. Let n_p the number of partitions $\lambda \vdash n$ that satisfies the previous requirements and α_i the number of *partition entries* $\lambda_i \in \lambda^{(i)} \vdash n$ that satisfies the monomial combination requirement. Then let $\mathcal{K}$ be the set that contains all pairs $(\lambda^{(i)} \vdash n, \alpha_i)$ such that $|\mathcal{K}| = n_p$. Then, the formulae now takes into account the number of matrices $(m \times n)$ with rank k:

$$\prod_{i=1}^{n_p} \alpha_i \prod_{j=1}^{|\lambda^{(i)}|} f(\lambda_i, \lambda_j, \min(\lambda_i, \lambda_j))$$

where $f(m,n,k)$ REF: the number of matrices $m \times n$ with rank $r \leq \min(m,n)$ over F_q is:

$$f(m,n,k) = q^{k(k-1)/2} \prod_{i=0}^{k-1} (q^{m-1}-1)(q^{n-i}-1)(q^{i+1}-1)^{-1}$$

4.5 Conclusion and results

It's been demonstrated that S must be a bijective affine transformation. If S is a linear transformation, there could be a chance that $\mathcal{A}(X)$ has invariant monomials found in polynomials $p_i(X) \in \mathcal{F}(X)$, this is equal to having equal submatrices in the representation $M_{\mathcal{A}(X)}$ and $M_{\mathcal{F}(X)}$ which leads the attacker to recover T. Thereof, an attacker could select arbitrary partitions $\lambda \vdash n$, then reorder the $M_{P(X)}$ and $M_{\mathcal{F}(X)}$ in terms of the new monomial basis he has selected, then test if $TM'_{F(X)} = M_{P(X)}$. If one of these overdetermined systems has a solution, he recovers T, reducing **IP2** to **IP1** by a Linear Algebra attack. These results are only applicable to schemes where the private polynomial $\mathcal{F}$ is known, concluding that S must be affine or if selected linear, it must not have the special form that has been addressed in the section for avoiding possible key recovery attacks.

Remark These results are experimental and not rigorous on their formal proof. The author's opinion is that there could be more matrices S of different forms that the given ones, plus the formulas to determine the number of matrices could be written in a total different way, as S must be a linear invertible matrix, there could be some S that have not complete rank, but are still counted in the formulae.

5 Cryptanalysis of Hidden Irreducible Polynomials

HIP is characterised by its initial phase on the scheme construction. For n odd:

$$\varphi : F_2^n \mapsto F_{2^n} \quad \varphi(c_0, \cdots, c_{n-1}) \mapsto \sum_{i=0}^{n-1} c_i x^i \tag{1}$$

$$\mathcal{F} : F_2[x] \times F_2[x] \mapsto F_{2^n} \quad \mathcal{F}(p(x), q(x)) \mapsto r(x) \tag{2}$$

$$P(x_1, \cdots, x_{n+1}) = T \circ \varphi^{-1} \circ \mathcal{F}(\quad \varphi(p(x)), \varphi(q(x)) \quad) \tag{3}$$

The map φ in (1) is a bijection from the vector space F_2^n to the field extension F_{2^n}, that's the polynomial quotient ring $\frac{F_2}{f(x)}$ where $f(x)$ is an irreducible polynomial of degree n over F_2. In (2) we observe that the function $\mathcal{F}$ multiplies two polynomials of degree $k-1$ where $k = \frac{n-1}{2} + 1$, obtaining a polynomial of degree $n-1$, this is a n-tuple when taken into the image of φ. Eventually, in (3) we notice that the public key map is the result of applying a linear bijective transformation $T : F_2^n \mapsto F_2^n$, obtaining a system of quadratic equations in $2k = n+1$ variables.

5.0.1 Public Key size and representation

The idea behind **HIP** is fancy at a first glance. The public key size is approximately half of the **HFE** public key size, concretely the size of the monomial basis of both schemes is given as:

$$|M_{B_{HIP}}| = (\frac{n-1}{2} + 1)^2 + 1$$

$$|M_{B_{HFE}}| = \binom{n+1}{2} + 1 = \frac{n(n-1)}{2} + 1$$

$$\lim_{n\to\infty} \frac{|M_{B_{HIP}}|}{|M_{B_{HFE}}|} = \frac{1}{2}$$

It's noticeable that $p_3(y) = \mathcal{F}(p_1(y), p_2(y))$ multiplies two polynomials thus the coefficient variables of p_1 are not mixed together, same for p_2. As a consequence, only coefficient variables of $p_1(x)$ and $p_2(x)$ are mixed giving k^2+1 monomials where $k = \frac{n-1}{2} + 1$. The monomial basis is composed of k^2+1 elements, so the public key $P(X)$ can be given as a $n \times |M_b|$ matrix. This basis representation is very important to succeed in the cryptanalysis later explained.

5.0.2 Making it secure by applying non-linear transformations

To fix the cryptosystem it's mandatory to hide the structure of the output polynomial $p_3(x) = \mathcal{F}(p_1(y), p_2(y))$. A naive idea would be to apply a non-linear transformation which would increase the monomial-degree thus greater public key bit size. For example, introducing an **HFE** polynomial $\mathcal{F}(X)$ as in section TODO NUM converts the quadratic monomials into quartics, thus requiring $\mathcal{O}(n \cdot \log_2 q \cdot (\binom{n+3}{4} + 1))$ bits instead of $\mathcal{O}(n \cdot log_2 q \cdot (\binom{n+1}{2} + 1))$.
Conclude that applying non-linear transformations to $p_3(y)$ implies an increase on the public key bit size, which instantly turns the scheme to be uninteresting because it overpasses the public key size of common schemes found in literature.

5.1 Cryptanalysis via matrix representation

In this section a full cryptanalysis of **HIP** is given. It follows similar ideas found in [2]. In addition, the techniques included in this paper aim to generalise the cryptanalysis to arbitrary parametrization over F_2, this is, it will always works so the current state of **HIP** is *completely broken*. It's important to note that $T \in F_q^n \mapsto F_q^n$ is a **linear** transformation and not an affine one, so the public key has size k^2 instead of k^2+1.

5.1.1 Overview

Recall that **HIP** has no non-linear functions beyond $\mathcal{F}$. Since we know the monomial basis found in the output $p_3(y)$ of $\mathcal{F}$ we can obtain the matrix representation of $\varphi(p_3(x))$ in the monomial basis.

The main principle of the attack is to find a **linear** transformation matrix T that transforms the matrix representation of $\mathcal{F}$ to the matrix representation of the public key in the monomial basis. Express this relation as an equation system:

$$
\begin{aligned}
&T \in F_q^{n\times n}, M_1, M_2 \in F_q^{n\times k^2}, \overrightarrow{M_b} \in F_q^{k^2\times 1}\\
&T \cdot M_1 \cdot \overrightarrow{M_b} = M_2 \cdot \overrightarrow{M_b}\\
&T \cdot M_1 = M_2
\end{aligned}
$$

The equation system holds by the principle that $T(\varphi(p_3(y))) = \overrightarrow{y}$ being equal to $T \cdot M_1 = M_2$ in the monomial basis. Mathematically, applying a linear transformation to a n-tuple of multivariate polynomials is the same as transforming the matrix representation of that n-tuple of multivariate polynomials in the monomial basis $\overrightarrow{M_b}$.

5.1.2 Generalisation

At this point, we've all the tools to recover the linear matrix transformation T. Let's define an instance of the cryptosystem:

Given the public key $P \in F_2^{n\times|M_b|}$ in the monomial basis $\overrightarrow{M_b}$ and $\overrightarrow{y} = P(X)$ find the input values $\overrightarrow{x} = (x_1, \cdots, x_{n+1})$ that represent two irreducible polynomials over F_2 with degree $\frac{n-1}{2}$.

First, define a map that transforms a n-tuple of multivariate polynomials into its matrix representation on the monomial basis of k^2 elements where $k = \frac{n-1}{2} + 1$.

$$\phi : F_2[x_1, \cdots, x_{n+1}]^n \mapsto F_2^{n\times k^2}, \quad (p_1(X), \cdots, p_n(X)) \mapsto M$$

where $M \cdot \overrightarrow{M_B} = (p_1(X). \cdots, p_n(X))$

The output vector $\overrightarrow{p_3} = \varphi(p_3(y)) = \mathcal{F}(p_1(y), p_2(y))$ has n multivariate polynomials in its coefficients, so $\overrightarrow{p_3} = (p_1(X), \cdots, p_n(X))$. Obtain M_1 by applying $M_1 = \phi(\overrightarrow{p_3})$. As the public key matrix P is given in the same monomial basis as M_1, rewrite $M_2 = P$ and obtain a new overdetermined system of linear equations as follows:

$$
\begin{aligned}
M_1 &= T^{-1} \cdot M_2\\
\overrightarrow{b_{M_1}} &= A \cdot \overrightarrow{x}
\end{aligned}
$$

Where $A \in F_2^{nM_b\times n^2}$ and $x \in F_2^{n^2\times 1}$ then clearly $b_{M_1} \in F_2^{nM_b\times 1}$ is the colum-vector representation of M_1. Solving the overdetermined system will always yield T^{-1} as we already know that M_2 is the transformation of M_1 by the transformation matrix T. Linear Algebra doesn't fail in this aspect. Finally, on $\overrightarrow{x}$ we obtain n^2 values, when reshaping to a $n \times n$ matrix gives T^{-1}.

5.2 Brief Example

Let $n = 7$ and $k = \frac{n-1}{2} + 1 = 4$. Then $p(x) = \sum_{i=1}^{k} x_i y^{i-1} \quad q(x) = \sum_{i=k+1}^{2k} x_i y^{i-1}$, both polynomials having degree $k - 1 = 3$, so $r(x)$ has degree 6. The number of variables is $n + 1 = 8$ as $p(x), q(x)$ both have $k+1 = 4$ variable coefficients. Then $r = \varphi^{-1} \circ \mathcal{F}(p(x), q(x))$ gives us the following n-tuple that's a symmetric triangular system of equations:

$$
\begin{gathered}
x_1x_5\\
x_2x_5 + x_1x_6\\
x_3x_5 + x_2x_6 + x_1x_7\\
x_4x_5 + x_3x_6 + x_2x_7 + x_1x_8\\
x_4x_6 + x_3x_7 + x_2x_8\\
x_4x_7 + x_3x_8\\
x_4x_8
\end{gathered}
$$

There are $k^2 = 16$ distinct monomials in $r \in F_2^n$. These are $\overrightarrow{M_B} = \{x_1x_5, x_1x_6, x_1x_7, x_1x_8, x_2x_5, x_2x_6, x_2x_7, x_2$
As we know $\overrightarrow{r}$ and the monomial basis, represent it as the matrix $M_r \in F_2^{n\times M_B}$ such that $\phi(r) = M_r$ in the monomial basis. Ending up in a 7×16 matrix where:

$$M_r \cdot \overrightarrow{M_B} = \overrightarrow{r}$$

$$M_r = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

Let $T \in F_2^{7\times 7}$ be the transformation matrix with entries:

$$T = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 \end{pmatrix}$$

As said before applying T to r as $T \cdot \overrightarrow{r} = \overrightarrow{y}$ is equal to $T \cdot \phi(r) = \phi(y)$ so $T \cdot M_r = M_y$ in the monomial basis. As a consequence the public key matrix representation is given by M_y in the monomial basis:

$$M_y = \begin{pmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 0 \end{pmatrix}$$

And the public key seen as a quadratic polynomial equation system in the monomial basis is:

$$P(X) = \begin{pmatrix} x_1x_8 + x_2x_7 + x_2x_8 + x_3x_6 + x_3x_7 + x_3x_8 + x_4x_5 + x_4x_6 + x_4x_7 + x_4x_8 \\ x_1x_7 + x_2x_6 + x_3x_5 + x_4x_8 \\ x_1x_5 + x_1x_7 + x_1x_8 + x_2x_6 + x_2x_7 + x_3x_5 + x_3x_6 + x_3x_8 + x_4x_5 + x_4x_7 \\ x_1x_5 + x_1x_7 + x_1x_8 + x_2x_6 + x_2x_7 + x_3x_5 + x_3x_6 + x_4x_5 + x_4x_8 \\ x_1x_6 + x_1x_7 + x_2x_5 + x_2x_6 + x_3x_5 + x_3x_8 + x_4x_7 \\ x_1x_6 + x_1x_7 + x_1x_8 + x_2x_5 + x_2x_6 + x_2x_7 + x_3x_5 + x_3x_6 + x_4x_5 \\ x_1x_6 + x_1x_7 + x_1x_8 + x_2x_5 + x_2x_6 + x_2x_7 + x_3x_5 + x_3x_6 + x_3x_8 + x_4x_5 + x_4x_7 \end{pmatrix}$$

We want to retrieve T^{-1} from $M_r = T^{-1}M_y$ so proceed to transform it to the system $\overrightarrow{b_{M_r}} = A \cdot \overrightarrow{x}$ where $\overrightarrow{b_{M_r}} \in F_2^{(7\times 16)\times 1}$, $A \in F_2^{7\cdot 16\times 49}$ and $\overrightarrow{x} \in F_2^{49\times 1}$. Using the CAS Mathematica we are able to solve this overdetermined system with ease. A screenshot from matrix A is included, as it's too big for being included as matrix in LaTeX.

$$
\begin{bmatrix}
1&1&1&1&1&0&0&0

Conclude that schemes based on transforming a previously known system of polynomial equations are **vulnerable**, as a consequence of knowing that M_1 and M_2 are given in the same monomial basis.

References

[1] Borja Gómez. *Hidden Irreducible Polynomials : A cryptosystem based on Multivariate Public Key Cryptography* https://eprint.iacr.org/2019/1174.pdf

[2] Christian Eder. *Breaking the Hidden Irreducible Polynomials Scheme* https://arxiv.org/pdf/1911.01830.pdf

[3] Christopher Wolf. *Multivariate Quadratic Polynomials in Public Key Cryptography* https://eprint.iacr.org/2005/393.pdf

[4] Jacques Patarin. *Hidden Fields Equations (HFE) and Isomorphisms of Polynomials (IP): Two New Families of Asymmetric Algorithms* https://link.springer.com/chapter/10.1007/3-540-68339-9_4

[5] Jacques Patarin. Louis Goubin. *Improved Algorithms for Isomorphisms of Polynomials* http://www.minrank.org/ip6long.ps

[6] Willi Geiselmann. Willi Meier. Rainer Steinwandt *An Attack on the Isomorphisms of Polynomials Problem with One Secret* https://eprint.iacr.org/2002/143.pdf

[7] Ludovic Perret. *A Fast Cryptanalysis of the Isomorphism of Polynomials with One Secret Problem* https://link.springer.com/chapter/10.1007/11426639_21

[8] Françoise Levy-Dit-Vehel. Ludovic Perret *Polynomial equivalence problems and applications to multivariate cryptosystems* https://hal.inria.fr/inria-00071464/document

[9] Aviad Kipnis. Jacques Patarin. Louis Goubin *Unbalanced Oil and Vinegar Signature Schemes* https://link.springer.com/chapter/10.1007/3-540-48910-X_15

[10] Aviad Kipnis. Adi Shamir *Cryptanalysis of the oil and vinegar signature scheme* https://link.springer.com/chapter/10.1007%2FBFb0055733